

Compliance & Secure Coding Practices on 25-26J-062

25-26J-062

InvisiGuard - Intelligent System for Predicting Cyber Threats in Real Time

NAWARATHNA M N I M	IT22343116
PALLEWATHTHA P A	IT22133304
NAWARATHNA N P D T	IT22117496
NETHKALUM G M H	IT22562906

Supervisor: Mr. Amila Senarathne

Date of Submission: 30/4/2026

1. INTRODUCTION

The Invisiguard, Real-Time Cyber Threat Prediction System is a cybersecurity product that detects both known and unknown cyber threats. It uses four components: system call pattern analysis, user behavior analysis, network traffic anomaly detection and endpoint log analysis. The project aims to improve early threat detection by spotting suspicious activities before they lead to serious damage. Compliance and secure coding practices were crucial during the implementation, as the system handles security-related data, logs, anomaly scores, model outputs and dashboard API responses. The secure coding practices used in the project included input validation, structured error handling, controlled API responses, secure logging, role-based access control and safe configuration management. These practices helped reduce risks such as unauthorized access, data leakage, incorrect model processing, insecure API usage and exposure of sensitive system details. As a result, secure coding enhanced both the reliability and security of the overall implementation.

2. SECURE CODING STANDARDS

During the development of this project, we followed secure coding practices to improve the safety, reliability and maintainability of the system. The primary standard we used to be the OWASP Secure Coding Practices, particularly for the web dashboard, backend APIs, data processing modules and component integration. The OWASP principles guided us in implementing input validation, secure authentication, safe error handling, access control, secure communication insecure direct object references and sensitive data exposure.

We also considered security principles from ISO/IEC 27001, especially regarding secure software development lifecycle practices, data protection, access control, documentation and risk management. Since the project focuses on predicting cybersecurity threats, maintaining confidentiality, integrity and availability of system data was a major concern. While the project mainly uses Python, machine learning libraries, APIs and a web dashboard, our secure development approach adhered to general industry best practices to ensure the system can be safely used in a real-time cybersecurity environment.

3. Key Secure Coding Techniques Implemented with Examples

3.1 Input Validation and Sanitization

All user inputs are validated using Pydantic models with strict type checking and field validation before processing across both backend systems.

```
class WindowEventSchema(BaseModel):
    event_id: int = Field(..., ge=1, le=65535)
    timestamp: datetime = Field(...)
    hostname: str = Field(..., min_length=1, max_length=255)
    event_type: str = Field(..., pattern=r'^(login|logout|file_access|device_load)$')

    @field_validator('hostname')
    @classmethod
    def validate_hostname(cls, v: str) -> str:
        if not HOSTNAME_PATTERN.match(v):
            raise ValueError("Invalid hostname format: [v]")
        return v
```

```
@validator("MAX_FPR_THRESHOLD")
def validate_fpr_threshold(cls, v):
    if v > 0.05:
        raise ValueError("MAX_FPR_THRESHOLD must be ≤0.05")
    return v
```

3.2 Authentication & Authorization

JWT-Based Authentication: Secure token-based authentication using the python-jose library with HS256 algorithm implemented across both backend systems.

```
def create_access_token(user_id: str, role: UserRole,
                        expires_delta: Optional[timedelta] = None) -> str:
    expire = datetime.utcnow() + (expires_delta or timedelta(hours=settings.JWT_EXPIRATION_HOURS))
    to_encode = {
        "sub": user_id,
        "role": role.value,
        "exp": expire,
        "iat": datetime.utcnow()
    }
    return jwt.encode(to_encode, settings.JWT_SECRET, algorithm=settings.JWT_ALGORITHM)
```

Role-Based Access Control (RBAC): Three-tier permission hierarchy (Viewer, Analyst, Admin) with granular access control.

```
def has_permission(self, required_role: UserRole) -> bool:
    role_hierarchy = {
        UserRole.VIEWER: 0,
        UserRole.ANALYST: 1,
        UserRole.ADMIN: 2
    }
    return role_hierarchy[self.role] >= role_hierarchy[required_role]
```

```
def has_permission(self, required_role: UserRole) -> bool:
    role_hierarchy = {
        UserRole.VIEWER: 0,
        UserRole.ANALYST: 1,
        UserRole.ADMIN: 2
    }
    return role_hierarchy[self.role] >= role_hierarchy[required_role]
```

Failed Login Tracking : Increments failed login attempts counter to detect brute force attacks.

3.3 Encryption

Password Hashing: Bcrypt algorithm with 12 rounds for secure password storage across both backend systems.

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def hash_password(plain_password: str) -> str:
    return pwd_context.hash(plain_password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)
```

Data-at-Rest Encryption: AES-256-GCM encryption via Fernet for sensitive data storage.

```
def encrypt_data(plaintext: str) -> str:
    cipher = get_fernet_cipher()
    encrypted = cipher.encrypt(plaintext.encode())
    return encrypted.decode()

def decrypt_data(ciphertext: str) -> str:
    cipher = get_fernet_cipher()
    decrypted = cipher.decrypt(ciphertext.encode())
    return decrypted.decode()
```

3.4 Error Handling and Logging

Secure Error Handling: Custom exception handlers prevent information leakage by returning generic error messages to clients while logging detailed errors internally.

```
credentials_exception = HTTPException(
    status_code=status.HTTP_401_UNAUTHORIZED,
    detail="Could not validate credentials",
    headers={"WWW-Authenticate": "Bearer"},
)
```

3.5 Audit Trail and Accountability

- **Immutable Audit Logging:** Comprehensive audit trail for all security-relevant actions with indefinite retention.

```
class AuditAction(enum.Enum):
    USER_LOGIN = "user_login"
    USER_LOGOUT = "user_logout"
    ENDPOINT_REGISTERED = "endpoint_registered"
    ALERT_CREATED = "alert_created"
    ALERT_ACKNOWLEDGED = "alert_acknowledged"
    ALERT_RESOLVED = "alert_resolved"
    MODEL_UPLOADED = "model_uploaded"
    MODEL_DEPLOYED = "model_deployed"
    DETECTION_FEEDBACK = "detection_feedback"
    CONFIG_CHANGED = "config_changed"
```

3.6 Secure Session Management

- **Token Expiration:** Configurable JWT expiration (default 1 hour) prevents token reuse attacks.
- **Active Account Validation:** User account status checked on every authenticated request
- **Bearer Token Security:** HTTP Bearer authentication scheme with proper header validation.

3.7 Rate Limiting and Performance Controls

Redis-Based Rate Limiting : Sliding window algorithm using Redis sorted sets for distributed rate limiting.

```
async def is_allowed(self, key: str, max_requests: int, window_seconds: int):
    # Use Redis sorted set with timestamps as scores
    pipe = self.redis_client.pipeline()
    pipe.zremrangebyscore(key, 0, window_start.timestamp())
    pipe.zcard(key)
    pipe.zadd(key, {str(now.timestamp()): now.timestamp()})
    pipe.expire(key, window_seconds)

    results = await pipe.execute()
    current_requests = results[1]
    is_allowed = current_requests < max_requests
```

4. Tools & Technologies Used for Secure Coding

The project uses a robust set of tools and technologies to ensure secure coding, reliability, and high performance. Security is enforced through libraries such as Passlib with bcrypt for strong password hashing, Cryptography (Fernet) for AES-256-GCM encryption, and Python-JOSE for secure JWT handling, alongside built-in protections from FastAPI. The development environment leverages Docker Compose for consistent deployments, with backend support from PostgreSQL, Redis, and Apache Kafka with Apache ZooKeeper.

Application components are built using Flask, while machine learning capabilities rely on TensorFlow, Keras, and scikit-learn. Network analysis is supported by tools like Wireshark and tshark. Additionally, code quality and security are maintained through Pydantic for runtime validation, SQLAlchemy to prevent SQL injection, and Python type hints for improved code safety and maintainability.

5. Compliance Standards

The project aligns with key compliance standards, primarily the General Data Protection Regulation (GDPR), while also considering frameworks such as SOC 2 Type II, and industry best practices like the OWASP Top 10 and OWASP Machine Learning Security guidelines. To ensure compliance, strict data protection measures are implemented, including data minimization, anonymization of personally identifiable information (PII) before machine learning processing, and the use of flow-based network analysis instead of raw packet inspection to preserve privacy. Data protection by design is enforced through encryption at rest and in transit, role-based access control (RBAC), and comprehensive audit logging. User rights such as the right to erasure are supported via account deactivation mechanisms, while user consent is managed and documented during system interactions. Additionally, well-defined data retention policies ensure regulatory adherence. Security best practices further strengthen compliance through protections against common vulnerabilities, secure coding techniques, and continuous monitoring of system and model integrity. Overall, the system integrates regulatory, security, and operational controls to maintain confidentiality, integrity, availability, and privacy of data in line with modern compliance requirements.

6. Challenges & Mitigation Strategies

One challenge was ensuring secure coding practices were applied consistently across all four project components: system call pattern analysis, user behavior analysis, network traffic anomaly detection and endpoint log analysis. Each component used different types of data, so input validation, preprocessing, and secure handling needed careful management. We addressed this by using structured validation with Pydantic, controlled feature processing, safe API responses and clear separation between frontend, backend, model and logging functions.

Another challenge was keeping sensitive technical details from being exposed during errors, failed API requests, or backend exceptions. Since the system relies on trained models, logs, alert files, and dashboard APIs, poor error handling could reveal file paths, system details, or internal logic. We handled this by using internal logging for developers, sanitized client error messages, protected frontend routes, role-based authorization, and MFA checks for admin-level access. These strategies helped improve confidentiality, reliability and security of the overall system.

7. References

- [1] OWASP, “OWASP Secure Coding Practices Quick Reference Guide,” [Online]. Available: <https://owasp.org/www-project-secure-coding-practices-quick-reference-guide/>
- [2] CERT, “CERT Secure Coding Standards,” [Online]. Available: <https://www.cert.org/secure-coding/>
- [3] MITRE, “CWE Top 25 Most Dangerous Software Weaknesses,” [Online]. Available: <https://cwe.mitre.org/top25/>
- [4] National Institute of Standards and Technology, “*NIST AI Risk Management Framework*,” [Online]. Available: <https://www.nist.gov/itl/ai-risk-management-framework>
- [5] OWASP, “*OWASP Machine Learning Security Top 10*,” [Online]. Available: <https://owasp.org/www-project-machine-learning-security-top-10/>
- [6] National Institute of Standards and Technology, “*NIST Cybersecurity Framework*,” [Online]. Available: <https://www.nist.gov/cyberframework>

